# Métodos de Reamostragem

Prof. Renan Gomes Vieira

Universidade Federal do Ceará - Campus Quixadá
Aprendizado de Máquina

2026.2

# Sumário

1 Métodos de Reamostragem

2 Validação Cruzada

# Métodos de Reamostragem

## Métodos de Reamostragem

Métodos que repetidamente coletam amostras de um conjunto de treino e **retreinam** o modelo de interesse para cada amostra para obter informação adicional acerca do modelo ajustado.

- Tais técnicas permitem obter informações acerca do modelo que não podem ser obtidas apenas usando a amostra original de treino.
- Essa técnicas tendem a ser computacionalmente caras.
- Vamos discutir a principal técnica em aprendizado de máquina: *validação-cruzada*.

## Relembrando

**Taxa de erro de teste** é o erro médio ao predizer amostras novas não usadas no treinamento do modelo.
**Taxa de erro de treino** é o erro médio ao predizer amostras usadas no treinamento do modelo.

- Apesar de tais valores serem fáceis de computar, geralmente essa separação de dados não está disponível.
- Existem métodos que ajustam o treino do erro para estimar o erro de teste (AIC, *Adjusted resubstitution*, BIC...).
- A forma mais comum é simplesmente *selecionar* um subconjunto dos dados de treino como de teste.

# Validação-Cruzada

Há três técnicas principais de avaliação de modelos:

- Conjunto de validação (*hold-out set)*
- *Leave-one-out cross-validation*
- *k-Fold cross-validation*

# Conjunto de validação (*hold-out set*)

## Conjunto de validação *Hold-out set*

Envolve separar *aleatoriamente* o conjunto de dados em duas partes: *conjunto de treino* e um *conjunto de validação*, também conhecido como *hold-out set*.

## Conjunto de teste ou conjunto de validação?

Comumente, ao aplicar uma validação *hold-out*, o conjunto não utilizado no treinamento é comumente chamado de **conjunto de teste**. Não há consenso sobre esses temos, e se diferem a depender do contexto, período de tempo e autor.

# Conjunto de validação (*hold-out set*)

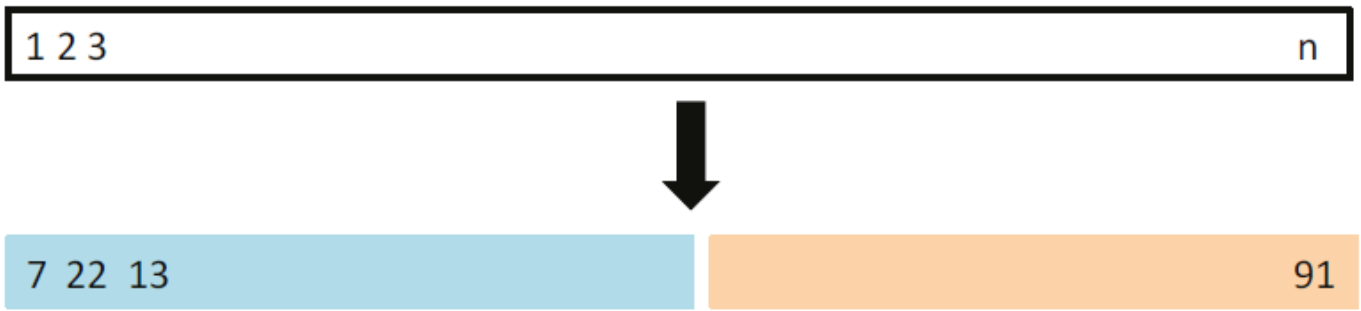



Figura 1: Esquema para mostrar a abordagem do conjunto de validação. Um conjunto de $n$ observações são aleatoriamente divididas em conjunto de treino (em azul) de validação (em laranja). Ajustamos o modelo com o conjunto de treino e avaliamos a performance no conjunto de validação.

# Conjunto de validação (*hold-out set*)

Não existe um consenso acerca do tamanho de tais conjuntos, mas há algumas regras de bolso (treino/teste), em percentual.

- 70/30 ou 80/20 - Recomendado com conjuntos de dados menores (na casa dos milhares) OU usar *k-fold*.
- 90/10 - Recomendado para conjuntos de dados maiores (centenas de milhares)

**Leituras Recomendadas**

Google e Scikit-Learn tem bons textos acerca do tema e essas divisões.

**O termo "Validação"**

O termo **validação** faz mais sentido quando consideramos que estamos usando-o para estimar hiperparâmetros do modelo.

# Conjunto de validação (*hold-out set*)

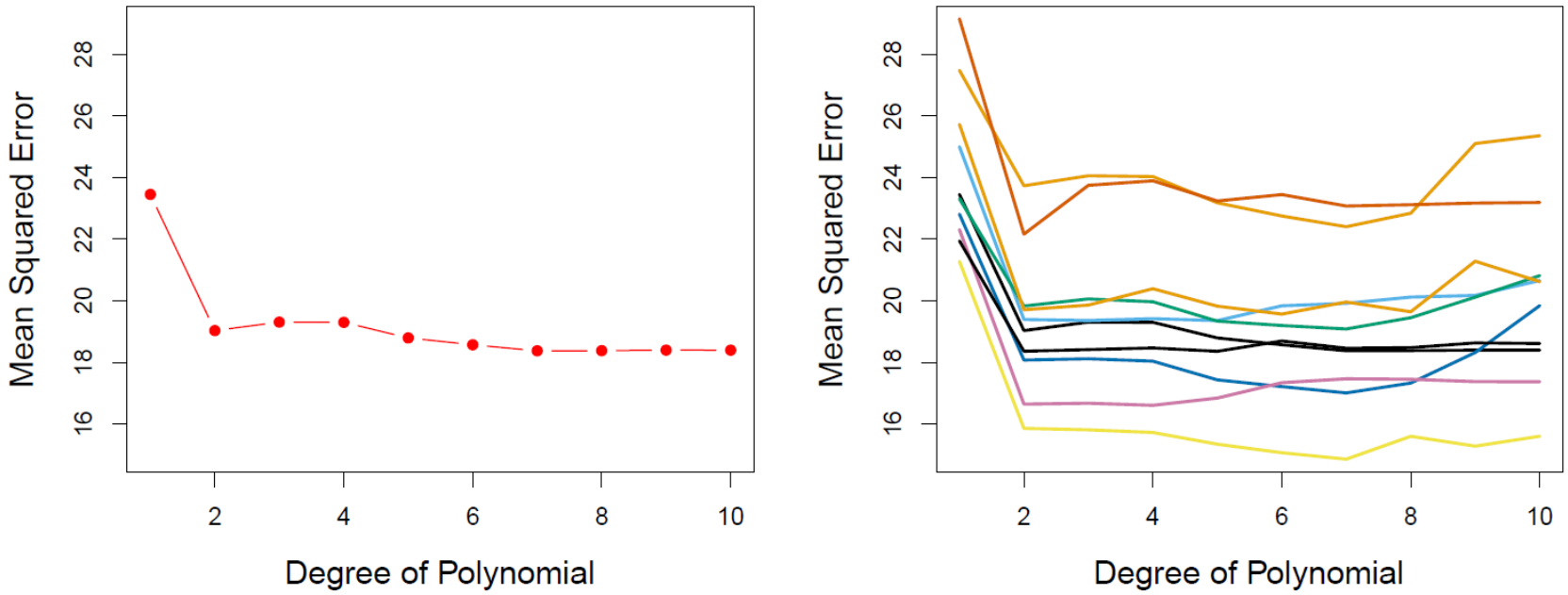



Figura 2: Abordagem do conjunto de validação no conjunto de dados Auto para estimar o erro de teste na predição de mpg com funções polinomiais de horsepower. À esquerda, as estimativas do erro de validação para uma única divisão entre treino e validação. À direita, a abordagem repetida dez vezes, cada uma com uma divisão aleatória diferente dos dados. Isso mostra a variabilidade da estimativa do EQM de teste gerada por essa abordagem.

# Conjunto de validação (*hold-out set*)

Observamos dois problemas com essa abordagem

1. A estimativa da validação do erro de teste pode ser altamente variável, dependendo de quais observações estão no conjunto de treino e quais estão no conjunto de validação.
2. Usamos somente um subconjunto das observações. Métodos estatísticos tendem a performar melhor quanto mais observações tiver.

Uma solução pra isso é usar métodos de *validação-cruzada*.

# Validação Cruzada

# *Leave-One-Out Cross-Validation*

- É uma aboragem muito similar ao *hold-out*, pois envolve dividir o cojunto em duas partes.
- Contudo, usamos apenas uma **única observação** $(x_1, y_1)$ como conjunto de validação e as observações restantes $\{(x_2, y_2), \cdots, (x_n, y_n)\}$ como conjunto de treino. Temos que

$$MSE_1 = (y_1 - \hat{y}_1)^2$$

  estima o erro de teste.
- Repetimos então o procedimento escolhendo $(x_2,y_2)$ como conjunto de validação e as observações restantes $\{(x_1, y_1), (x_3, y_3), \cdots, (x_n, y_n)\}$ como treino e computamos

$$MSE_2 = (y_2 - \hat{y}_2)^2$$

- Repetimos o processo $n$ vezes, para diferentes $i$'s, e computamos

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^{n} MSE_i$$

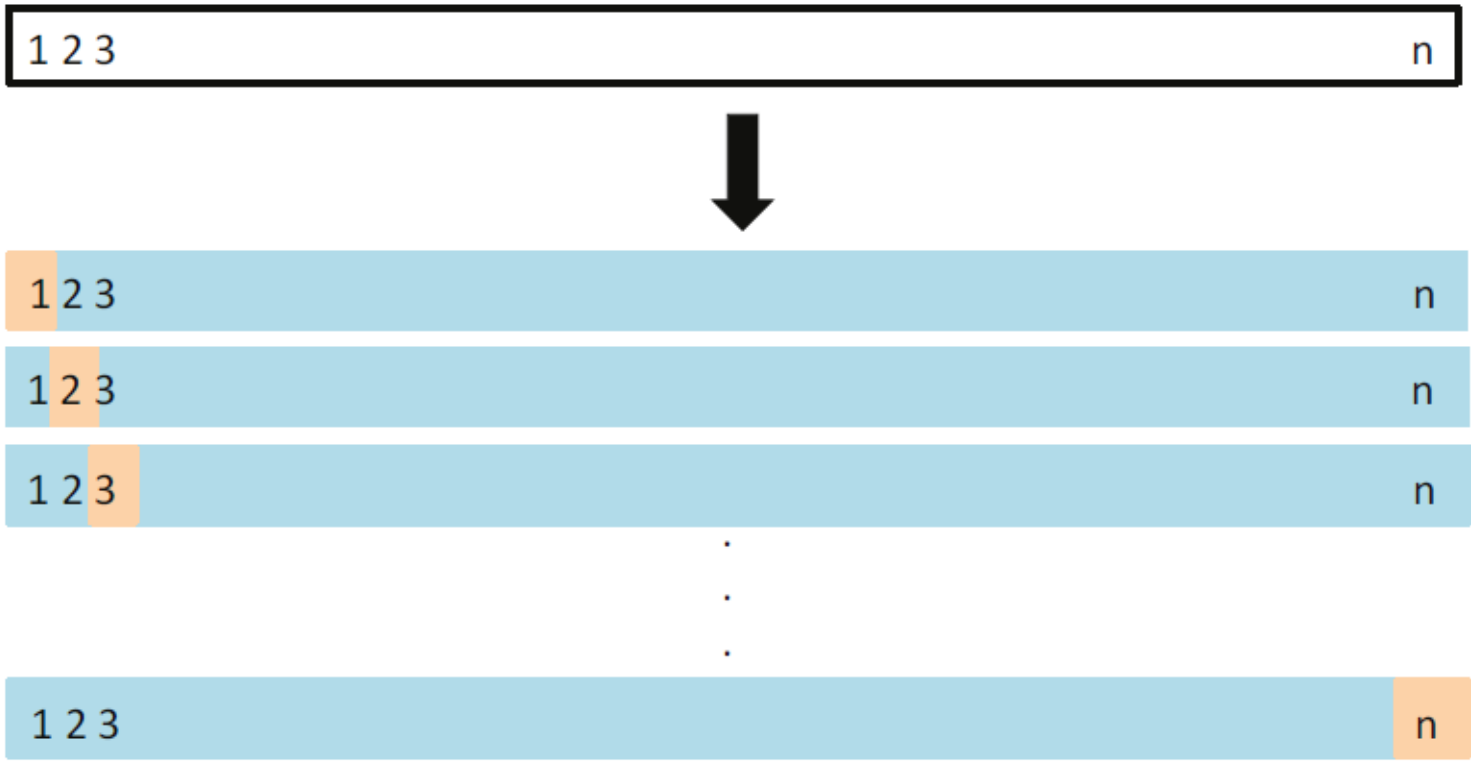



Figura 3: Esquema visual do LOOCV. Um conjunto de $n$ dados é repetidamente dividido em treino, contendo todas as observações, exceto uma, que é considerado como cojunto de validacão. O erro de teste é computado pegando a média do erro dos $n$ MSEs resultados de cada interação.

# *Leave-One-Out Cross-Validation*

Dessa forma, resolvemos os dois problemas do *hold-out*

- Todo conjunto de treino tem quase todas as observações, implicando em menos viés
- Exclui a aleatoriedade em escolher as divisões treino/validação.

É uma abordagem que funciona muito bem para qualquer modelagem.

**? Pergunta**

Mas qual o problema desta abordagem?

## *k-Fold Cross-Validation*

É uma alternativa ao LOOCV, que envolve dividir aleatoriamente o grupo de observações em $k$ grupos, também conhecido como *folds*, de aproximadamente mesmo tamanho.

- A cada interação, cada *fold* $i$ é tratado como conjunto de validação e o conjunto de treino são os $k-1$ *folds*, e computamos o $MSE_i$.
- O procedimento é repetido $k$ vezes, cada vez tendo como conjunto de validação é diferente.
- A estimativa de $k$-fold é computada pela media dos $MSE_i$,

$$CV_{(k)} = \frac{1}{k}\sum_{i=1}^{k} MSE_i$$

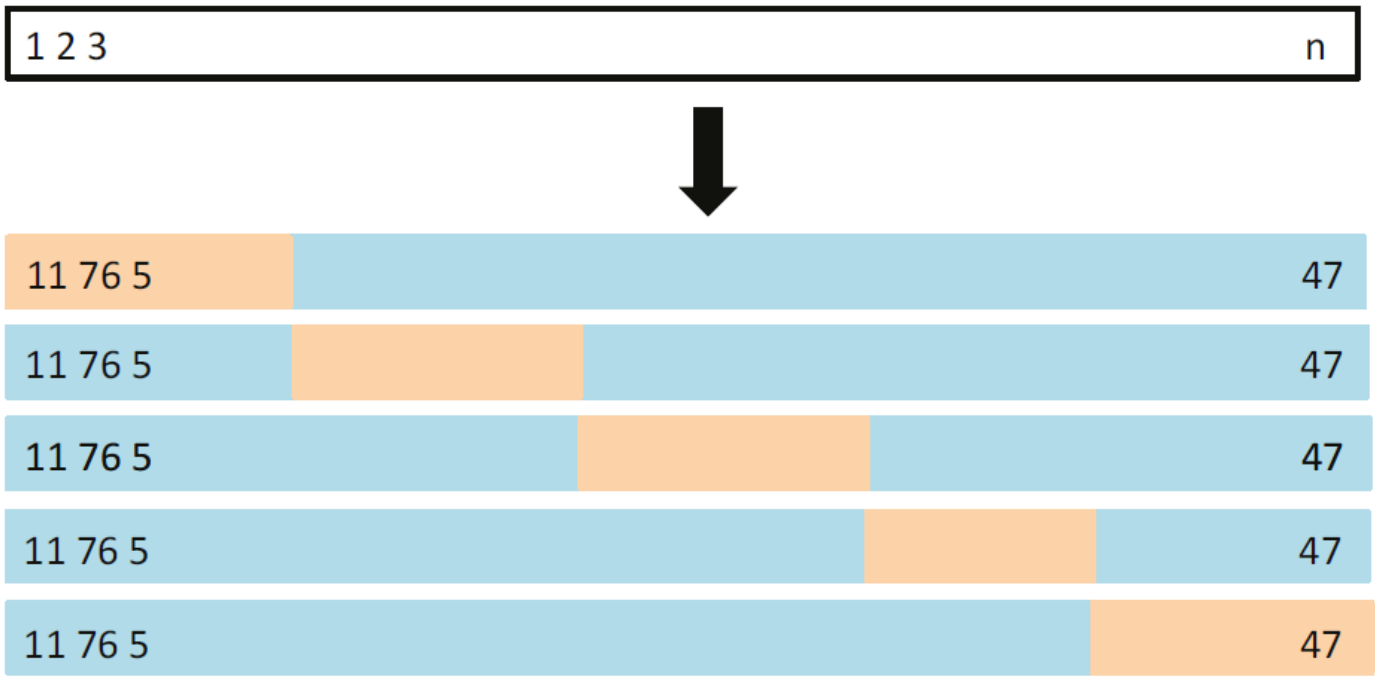



Figura 4: Esquema gráfico de um 5-*fold* CV. Um conjunto de $n$ observações é aleatoriamente dividido em cinco grupos disjuntos. Cada um desses subgrupos são utilizados como conjuntos de validação em 5 modelagens diferentes, enquanto seus complementos são usados para ajustar o modelo. O erro de teste é estimado ao calcular a média de seus respectivos MSE.

# *k-Fold Cross-Validation*

- Observe que o LOOCV é um caso particular do *k-fold* com $k = n$.
- Valores comuns de $k$ são 5 ou 10.
- A principal vantagem está relacionado o menor custo computacional.
- Método que pode ser aplicado na maioria dos métodos de aprendizado de máquina.
- Mas.. como ele se compara ao LOOCV?

# LOOCV vs. k-*Fold*

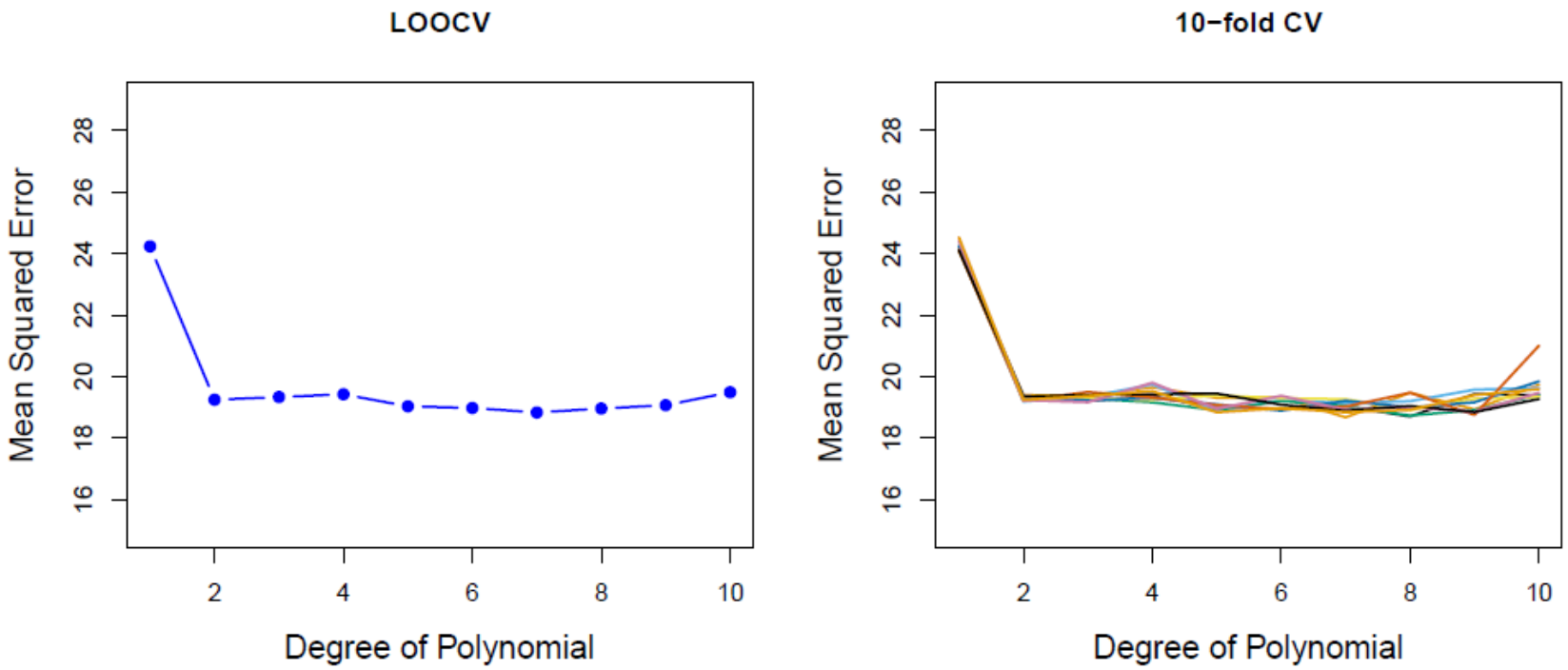



Figura 5: Validação cruzada usada para estimar o erro de teste usando diferentes funções polinomias de diferentes graus. Na esquerda, o *LOOCV* e a direita o 10-*fold* CV, executado nove vezes diferentes, com diferentes divisões treino/validação. Observe como as curvas são parecidas entre si e com a curva do LOOCV.

# Validação Cruzada

- Como sabemos se a estimativa da validação cruzada é apropriada?
- Podemos avaliar isso apenas com dados simulados, e observamos que o desempenho da validação cruzada varia a depender dos dados

**Não importa o valor real do MSE**

Uma discussão importante é perceber que por mais que o as estimativas dos valores *reais* possam ser erradas, o *local mínimo da curva do MSE* é o que importa.

# *Cross-Validation* e MSE real de teste

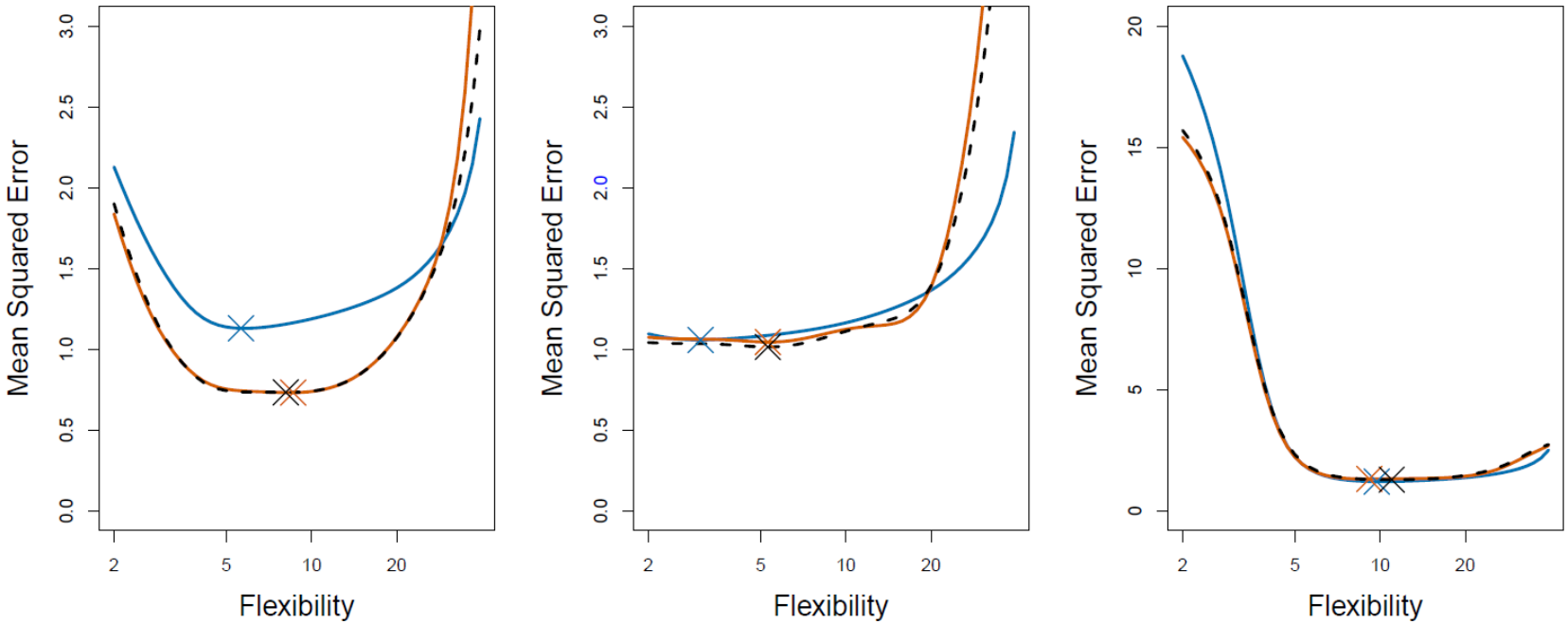



Figura 6: MSE de teste verdadeiro e estimado via validação-cruzada para três conjuntos de dados diferentes. O verdadeiro MSE está em azul, o LOCCV é a linha preta tracejada e o 10-*fold* é a linha em laranja. O X aponta o mínimo de cada curva de MSE.

# *Trade-off* Viés-Variancia do $k$-Fold CV

- Vimos que *k-fold* com $k < n$ possui uma vantagem computacional em relação ao LOOCV.

**Pergunta**

Quem produz melhores estimativas da taxa de erro de teste? *Hold-out*, LOOCV ou *k-fold*? Como o valor de $k$ está relacionado com o *trade-off* de viés-variância?

- Empiricamente, já é consolidado que *k-fold* usando $k = 5$ ou $k = 10$ estimam valores razoáveis de erro de teste que não sofrem muito nem de viés nem de variância.

# Validação-cruzada (CV) em problemas de classificação

Para classificação, a abordagem é muito similar, basta mudarmos a métrica de avaliação. Por exemplo, usando LOOCV

$$CV_{(n)} = \frac{1}{n} \sum_{i=1}^{n} Err_i$$

onde $Err_i = I(y_i \neq \hat{y}_i)$. *Hold-out* e *k-fold* são definidos de forma análoga.

# CV para classificação

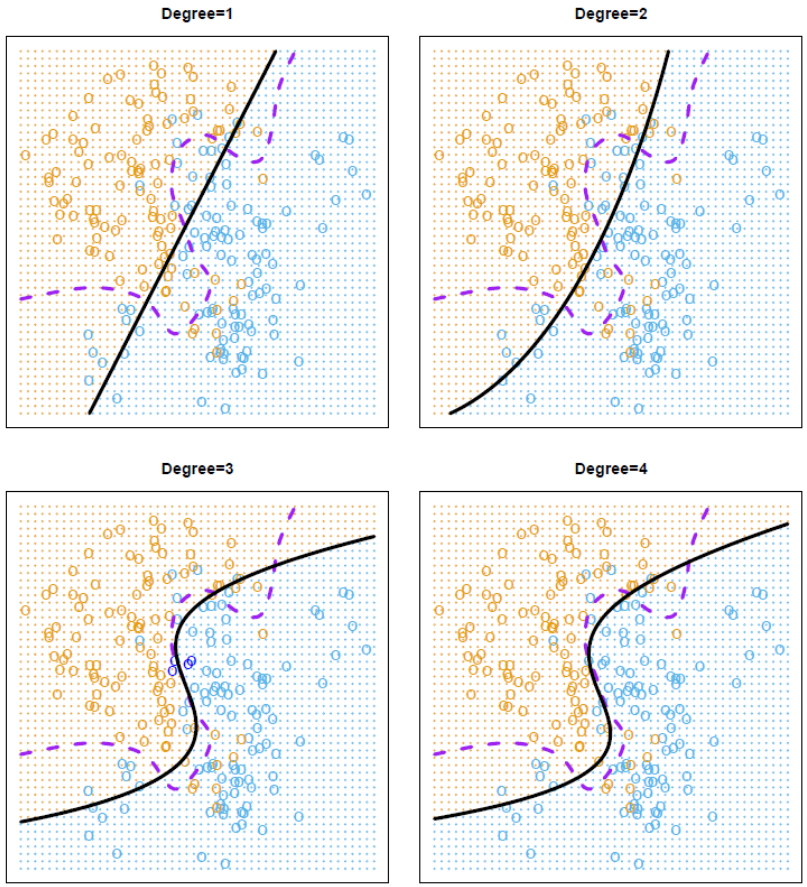



Figura 7: Regressão logística ajustado em dados de classificação bidimensional. A fronteira de decisão de Bayes é a linha tracejada roxa. Diferentes estimativas (lineares, quadrática, cúbica e quártica) são descritas em preto. O erros são: 0.201, 0.197, 0.160 e 0.162, enquanto o erro de Bayes é 0.133.

# CV para classificação

- Lembrando que o erro de teste e a fronteira de decisão de Bayes são desconhecidos em um cenário real.
- Mais uma vez, para decidir entre diferentes possíveis modelos, usamos a validação cruzada.
- Mais uma vez, estamos interessados em observar se o valor de menor erro para a validação cruzada é próximo do menor erro de teste real em algum hiperparâmetro do modelo a ser escolhido.

# CV para classificação

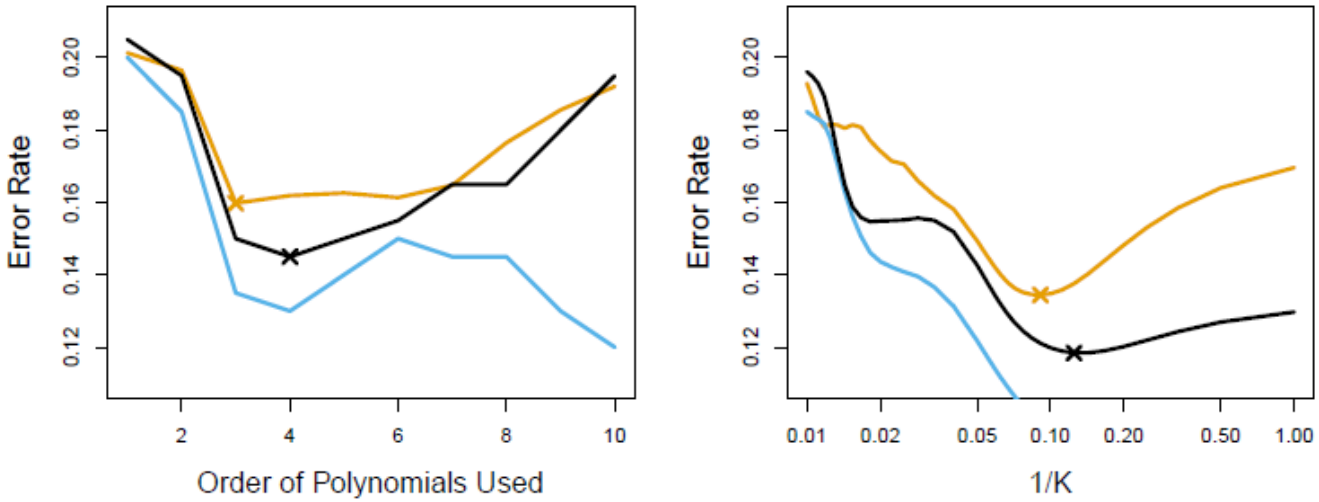



Figura 8: Erro de teste real (amarelo), erro de treino (azul) e o erro 10-*fold* CV, em dados de classificação. Na esquerda, regressão logistica usando diferentes funções polinomiais (eixo-y) dos preditores. Na direita, o classificador KNN com diferentes valores de K, o número de vizinhos.

# Leitura Recomendada

[1] Capítulo 5

# Referências I


Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor.
An Introduction to Statistical Learning with Applications in Python.
Springer Texts in Statistics. Springer, Cham, 1st edition, 2023.